Computer Vision and Deep Learning

Week 1 Manual

Introduction to Computer Vision and Deep Learning

Learning Objectives

By the end of this week, you will:

- Understand basic image processing concepts and operations
- Work with images programmatically using Python
- Grasp fundamental neural network concepts
- Implement a simple neural network from scratch
- Apply basic computer vision techniques to real problems

Prerequisites:

- Basic Python programming knowledge
- High school level mathematics (algebra, basic calculus)
- Familiarity with NumPy (we'll review as needed)

June 24, 2025

Contents

1 Assignment 1: Image Processing Fundamentals

Duration: Monday - Tuesday — **Assessment:** Tuesday End of Day

1.1 Learning Material

1.1.1 What is Computer Vision?

Computer Vision is a field of artificial intelligence that enables computers to interpret and understand visual information from the world. It involves:

- **Image acquisition**: Capturing or loading digital images
- **Image processing**: Manipulating images to enhance or extract information
- **Feature extraction**: Identifying important patterns or characteristics
- **Object recognition**: Identifying and classifying objects in images

1.1.2 Digital Images: The Foundation

A digital image is essentially a matrix of numbers:

- **Grayscale images**: 2D array where each value represents pixel intensity (0-255)
- **Color images**: 3D array with three channels (Red, Green, Blue)
- **Image coordinates**: (x, y) where (0,0) is typically top-left corner

1.1.3 Basic Image Operations

1. Loading and displaying images
2. Converting between color spaces (RGB, Grayscale, HSV)
3. Basic transformations (resize, rotate, flip)
4. Pixel manipulation and intensity adjustments
5. Filtering (blur, sharpen, edge detection)

1.2 Demo Module

1.2.1 Setup and Basic Operations

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load an image
img = cv2.imread('sample_image.jpg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  # OpenCV uses BGR by default

# Display image
plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title('Original Image')
plt.axis('off')

# Convert to grayscale
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.subplot(1, 2, 2)
plt.imshow(gray_img, cmap='gray')
plt.title('Grayscale Image')
plt.axis('off')
```

```
22 plt.show()
```

Listing 1: Basic Image Operations Demo

Expected Output: Side-by-side display of original color image and its grayscale version, demonstrating understanding of how color information is converted to intensity values.

1.3 Task Modules

1.3.1 Task 1.1: Image Exploration (Monday)

Create a Python script that:

1. Loads an image of your choice
2. Prints the image dimensions and data type
3. Displays the image in color and grayscale
4. Shows a histogram of pixel intensities for the grayscale version
5. Extracts and displays each color channel (R, G, B) separately

Deliverable: Python script + output images showing original, grayscale, histogram, and RGB channels

1.3.2 Task 1.2: Basic Image Transformations (Tuesday)

Implement the following transformations:

1. Resize image to 50% of original size
2. Rotate image by 45 degrees
3. Apply horizontal and vertical flips
4. Adjust brightness (+50) and contrast ($\times 1.5$)
5. Apply Gaussian blur with kernel size 15

Deliverable: Python script + grid showing all transformations applied to your image

1.4 Assessment Criteria for Assignment 1

- **Code Quality (40%):** Clean, commented, working code
- **Understanding (30%):** Correct implementation of image operations
- **Visualization (20%):** Clear, well-labeled output images
- **Creativity (10%):** Choice of interesting input image and presentation

2 Assignment 2: Neural Networks and Deep Learning Basics

Duration: Wednesday - Thursday — **Assessment:** Thursday End of Day

2.1 Learning Material

2.1.1 What are Neural Networks?

Neural networks are computing systems inspired by biological neural networks. They consist of:

- **Neurons (nodes):** Basic processing units that receive inputs and produce outputs
- **Weights:** Parameters that determine the strength of connections
- **Activation functions:** Mathematical functions that determine neuron output
- **Layers:** Groups of neurons (input, hidden, output layers)

2.1.2 Key Concepts

1. **Forward propagation:** Data flows from input to output
2. **Loss function:** Measures how wrong our predictions are
3. **Backpropagation:** Algorithm to update weights based on errors
4. **Training:** Process of adjusting weights to minimize loss

2.1.3 The Perceptron: Simplest Neural Network

A single neuron that:

- Takes multiple inputs
- Multiplies each by a weight
- Sums the results
- Applies an activation function
- Produces an output

Mathematical representation:

$$\text{output} = \text{activation\_function}\left(\sum(\text{inputs} \times \text{weights}) + \text{bias}\right) \tag{1}$$

2.2 Demo Module

2.2.1 Simple Perceptron Implementation

```python
import numpy as np
import matplotlib.pyplot as plt

class SimplePerceptron:
    def __init__(self, input_size):
        # Initialize weights randomly
        self.weights = np.random.random(input_size) * 0.1
        self.bias = np.random.random() * 0.1

    def sigmoid(self, x):
        """Sigmoid activation function"""
        return 1 / (1 + np.exp(-np.clip(x, -500, 500)))  # Clip to prevent
    overflow

    def forward(self, inputs):
        """Forward propagation"""
```

```
        weighted_sum = np.dot(inputs, self.weights) + self.bias
        return self.sigmoid(weighted_sum)

    def train_step(self, inputs, target, learning_rate=0.1):
        """Single training step"""
        # Forward pass
        prediction = self.forward(inputs)

        # Calculate error
        error = target - prediction

        # Update weights and bias
        self.weights += learning_rate * error * prediction * (1 - prediction) *
    inputs
        self.bias += learning_rate * error * prediction * (1 - prediction)

        return error

# Example: Training to learn logical AND operation
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])  # AND gate truth table

perceptron = SimplePerceptron(2)
errors = []

# Training loop
for epoch in range(1000):
    epoch_error = 0
    for i in range(len(X)):
        error = perceptron.train_step(X[i], y[i])
        epoch_error += abs(error)
    errors.append(epoch_error)

# Test the trained perceptron
print("Testing trained perceptron on AND gate:")
for i in range(len(X)):
    prediction = perceptron.forward(X[i])
    print(f"Input: {X[i]}, Target: {y[i]}, Prediction: {prediction:.3f}")
```

Listing 2: Simple Perceptron Implementation

Expected Output: Decreasing error over training epochs and final predictions close to target values for AND gate logic.

2.3 Task Modules

2.3.1 Task 2.1: Understanding Neural Network Components (Wednesday)

1. Implement different activation functions: Sigmoid, ReLU, Tanh
2. Plot their graphs and derivatives
3. Create a simple dataset (XOR problem: inputs [0,0], [0,1], [1,0], [1,1] with outputs [0,1,1,0])
4. Visualize why a single perceptron cannot solve XOR (hint: it's not linearly separable)

Deliverable: Python script with activation function plots and explanation of XOR problem

2.3.2 Task 2.2: Multi-Layer Neural Network (Thursday)

Build a simple 2-layer neural network to solve the XOR problem:

1. Create a network with 2 input neurons, 2 hidden neurons, 1 output neuron

2. Implement forward propagation through both layers
3. Train the network using backpropagation
4. Plot training loss over time
5. Test final accuracy on XOR problem

Deliverable: Working neural network code + training plots + accuracy results

2.4 Assessment Criteria for Assignment 2

- **Implementation (50%):** Correct neural network implementation
- **Understanding (25%):** Demonstration of key concepts through code
- **Analysis (15%):** Training plots and performance analysis
- **Problem Solving (10%):** Successfully solving XOR problem

3 Weekly Assessment (Friday)

Comprehensive evaluation covering both assignments

The weekly assessment will combine concepts from both assignments, requiring students to demonstrate their understanding of image processing fundamentals and basic neural network implementation through practical applications.

4 Resources and Tools

4.1 Required Software

- Python 3.7+
- OpenCV (`pip install opencv-python`)
- NumPy (`pip install numpy`)
- Matplotlib (`pip install matplotlib`)
- Jupyter Notebook (recommended)

4.2 Recommended Reading

- "Computer Vision: Algorithms and Applications" by Richard Szeliski (Chapters 1-2)
- "Deep Learning" by Ian Goodfellow (Chapter 1)
- Online: OpenCV Python tutorials
- Online: 3Blue1Brown Neural Networks series (YouTube)

4.3 Sample Datasets

- Use any personal photos for image processing tasks
- MNIST digits dataset (for neural network practice)
- Simple geometric shapes you can create programmatically

5 Next Week Preview

Week 2 will cover:

- Convolutional Neural Networks (CNNs)
- Feature detection and extraction
- Introduction to popular deep learning frameworks (PyTorch/TensorFlow)

Good luck with your first week! Remember, everyone learns at their own pace. Focus on understanding concepts rather than just completing tasks.